Site configuration

Website settings live in `calepin.toml` at the root of your website source directory. Use `--config <path>` only when the config file lives somewhere else.

Basic settings

A small site can start with:

```toml
# Site name shown by bundled themes and page metadata.
# Default: none.
title = "My Site"

# Short site description for metadata and previews.
# Default: none.
description = "A static website built from Typst documents."

# Public site URL, used for sitemap.xml and canonical URLs.
# Default: none.
base_url = "https://example.com"

# Theme: "calepin", "academic", a local theme directory, or false for raw output.
# Default: "calepin".
theme = "calepin"

# Render .pdf files for every page.
# Default: false.
pdf = false

# Minify generated HTML, including inline CSS and JavaScript.
# Default: false.
minify = true

# Static search engine. Set to "pagefind" to generate a search index.
# Default: none.
search = "pagefind"

# Logo image path for the top bar.
# Default: none.
logo = "assets/logo.svg"

# Alternative text for the logo image. Defaults to `title` when omitted.
# Default: none.
logo_alt = "My Site"

# Output directory for browser-facing generated assets.
# Also controls where the Calepin runtime is written.
# Default: ".calepin".
asset-dir = "_calepin"

# Browser favicon path. Omit this to use Calepin's generated default.
# Default: `asset-dir`/favicon.svg.
favicon = "assets/favicon.ico"

# HTML syntax highlighting themes. Paths are relative to calepin.toml.
# Default: built-in Calepin light and dark themes.
highlight-light = "themes/syntax/light.tmTheme"
highlight-dark = "themes/syntax/dark.tmTheme"
```

Use `--format html` for a one-time HTML-only build, regardless of `pdf`. Use `--minify` to minify HTML for a single build without changing `calepin.toml`.

Set `search = "pagefind"` to add static search to bundled website themes. *Calepin* writes the Pagefind search bundle to `pagefind/` after rendering the site and links the bundled themes to the generated component script and stylesheet. Bundled themes mark the main page content with `data-pagefind-body`, so navigation, toolbars, and footers are excluded from the search index.

Paths in `calepin.toml` are interpreted from the website source directory: the directory that contains the config file, unless you pass an explicit `--config`. This includes `logo`, `favicon`, page `target` values ending in `.typ`, page `glob` patterns, page and static `include` and `exclude` patterns, and local theme paths. During the build, *Calepin* rewrites internal files and generated assets as page-relative URLs, so links and images continue to work from nested pages. Literal URLs such as `https://example.com` are left unchanged.

Auto-generated metadata

Bundled website themes emit a small set of page-level head tags from config:

- `<title>` (fallback when no `<title>` is present in the rendered page)
- `<meta name="description">` and `<meta property="og:description">` from `description`

- `<meta property="og:title">` from page title and site title
- `<meta property="og:site_name">` from `title`
- `<meta property="og:url">` and `<link rel="canonical">` from `base_url` plus the page route

If you inject your own tags for these fields in a theme override, check for duplication with what the theme already emits.

Theme customization

Use a local theme when you want project CSS or template overrides. Point `theme` at the local theme directory and declare the bundled theme you want to extend in `theme.toml`:

```toml
# calepin.toml
theme = "themes/my-site"
```

```toml
# themes/my-site/theme.toml
extends = "academic"
```

Place project CSS in `themes/my-site/css/`. Local theme styles load after the parent theme's shared and local styles, so they can adjust colors, fonts, spacing, or component rules. The same theme directory can override HTML layouts, partials, scripts, and `layouts/pdf.typ`.

See Themes for the stable `--calepin-*` CSS tokens exposed by bundled themes.

Paths inside `.typ` files follow Typst path rules, not `calepin.toml` rules. In website builds, use root-relative Typst paths for shared website assets, such as `#image("/assets/diagram.svg")`; the leading `/` points at the website source directory, so the same source works from pages in subdirectories. Avoid bare relative paths such as `#image("assets/diagram.svg")` for shared assets in nested pages. If no favicon is set, *Calepin* writes a small default to `asset-dir`/favicon.svg; if no logo is set, bundled themes use `title` as the site name.

Syntax highlighting

Configure HTML syntax highlighting with full TextMate `.tmTheme` files:

```toml
highlight-light = "themes/syntax/light.tmTheme"
highlight-dark = "themes/syntax/dark.tmTheme"
```

Paths are resolved relative to `calepin.toml`. The light theme is used for light browser mode, and the dark theme is used for dark browser mode.

Paged output is different: PDF, SVG, and PNG are rendered directly by Typst, so they use Typst's standard `raw` highlighting. If you want a fixed paged syntax theme, customize it in Typst:

```typst
#let paper-code-theme = read("themes/syntax/paper.tmTheme", encoding: none)

#show raw.where(block: true): it => {
  raw(it.text, block: true, lang: it.lang, theme: paper-code-theme)
}
```

For HTML, *Calepin* keeps syntax tokenization consistent with Typst. Typst still reads Sublime syntax definitions (`.sublime-syntax`) and highlights the code; *Calepin* then maps the generated HTML colors to CSS classes so the final colors can depend on light or dark mode.

The HTML mapping uses the TextMate settings that affect rendered spans: `foreground`, `background`, and `fontStyle` values such as `bold`, `italic`, and `underline`. Use light and dark themes with similar scope rules for the most predictable result. If a scope exists in one theme but not the other, *Calepin* falls back to that theme's global foreground.

Robots.txt

Calepin writes `robots.txt` by default:

```
User-agent: *
Allow: /
Sitemap: https://example.com/sitemap.xml
```

The `Sitemap:` line is included when `base_url` is set. To disable generation:

```
robots = false
```

or:

```
[robots]
enabled = false
```

To override the generated file, create `templates/robots.txt` in the website source directory. The template uses MiniJinja syntax and receives `config` and `sitemap_url`:

```
User-agent: *
Disallow: /drafts/
{% if sitemap_url %}Sitemap: {{ sitemap_url }}{% endif %}
```

Files under `templates/` can be included or extended from the robots template:

```
{% extends "base.txt" %}
{% block body %}
User-agent: *
Allow: /
{% endblock %}
```

Static files

Use `[static]` for files that should be copied to the built website without being rendered as Typst pages:

```
[static]
include = [
  "assets/**",
  "CNAME",
  "downloads/**",
]
exclude = [
  "assets/drafts/**",
]
```

Each included file keeps its source-relative path in the output directory: `assets/logo.svg` becomes `assets/logo.svg`, `CNAME` becomes `CNAME`, and `downloads/manual.pdf` becomes `downloads/manual.pdf`. `include` entries can be files, directories, or glob patterns. `exclude` patterns are applied after includes. Paths must stay inside the website source directory.